CINECA AGENTIC PLATFORM

Enterprise-Grade AI Agent Orchestration Platform Connecting Multi-Tenant Architecture, MCP Tools, and Secure Graph Querying

Author:
Arman Feili

University:
Sapienza University of Rome

Advisor:
Prof. Marco Raoul Marini

Department:
Information Engineering, Electronics and Telecommunications

Co-Advisors:
Dr. Valerio Venanzi
Dr. Giuseppe Melfi
Dr. Marco Puccini

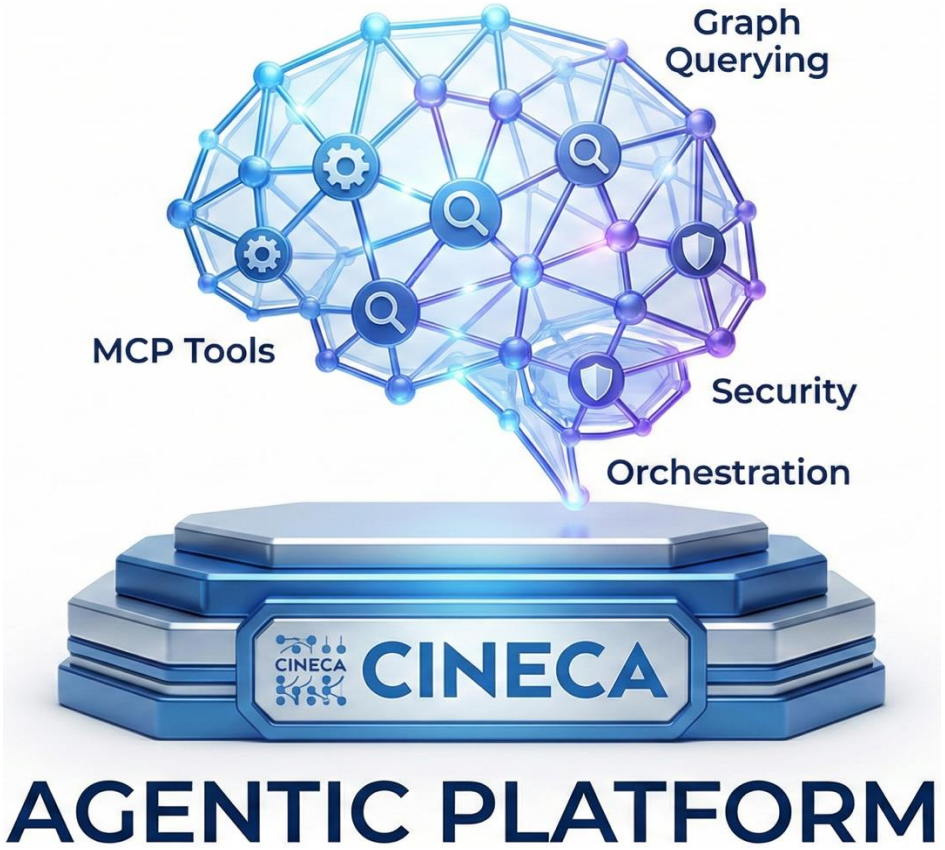
Company:
CINECA

Academic Year:
2025/2026

SAPIENZA
UNIVERSITÀ DI ROMA

CINECA

INDEX



PRODUCTION GAP

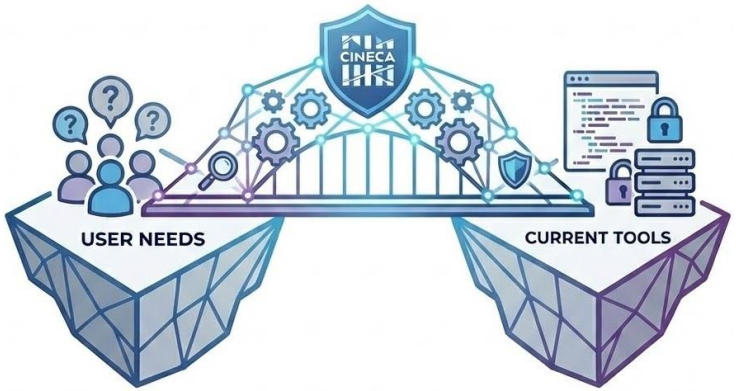
The gap between what users need and what current tools provide.

The Problem

- Most users can’t write Cypher, so they rely on developers to access and analyze graph data. This creates delays and technical bottlenecks, data stays locked behind complexity.

Why Simple Chatbots Don’t Work

- Basic chatbots can't handle real tasks.
- They:
  - Only answer one question at a time, no multi-step planning.
  - Can’t query databases or run actual jobs.
  - Lack security (no RBAC or tenant isolation).
  - Don’t log actions, no audit trail.
  - Produce inconsistent outputs, no reproducibility or traceability.

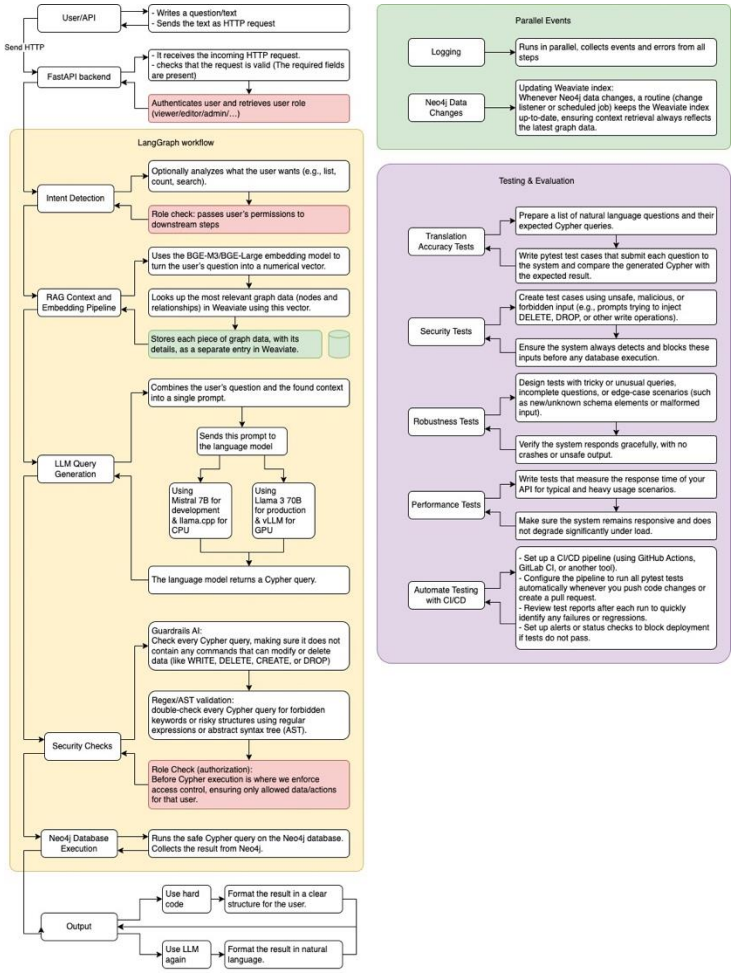


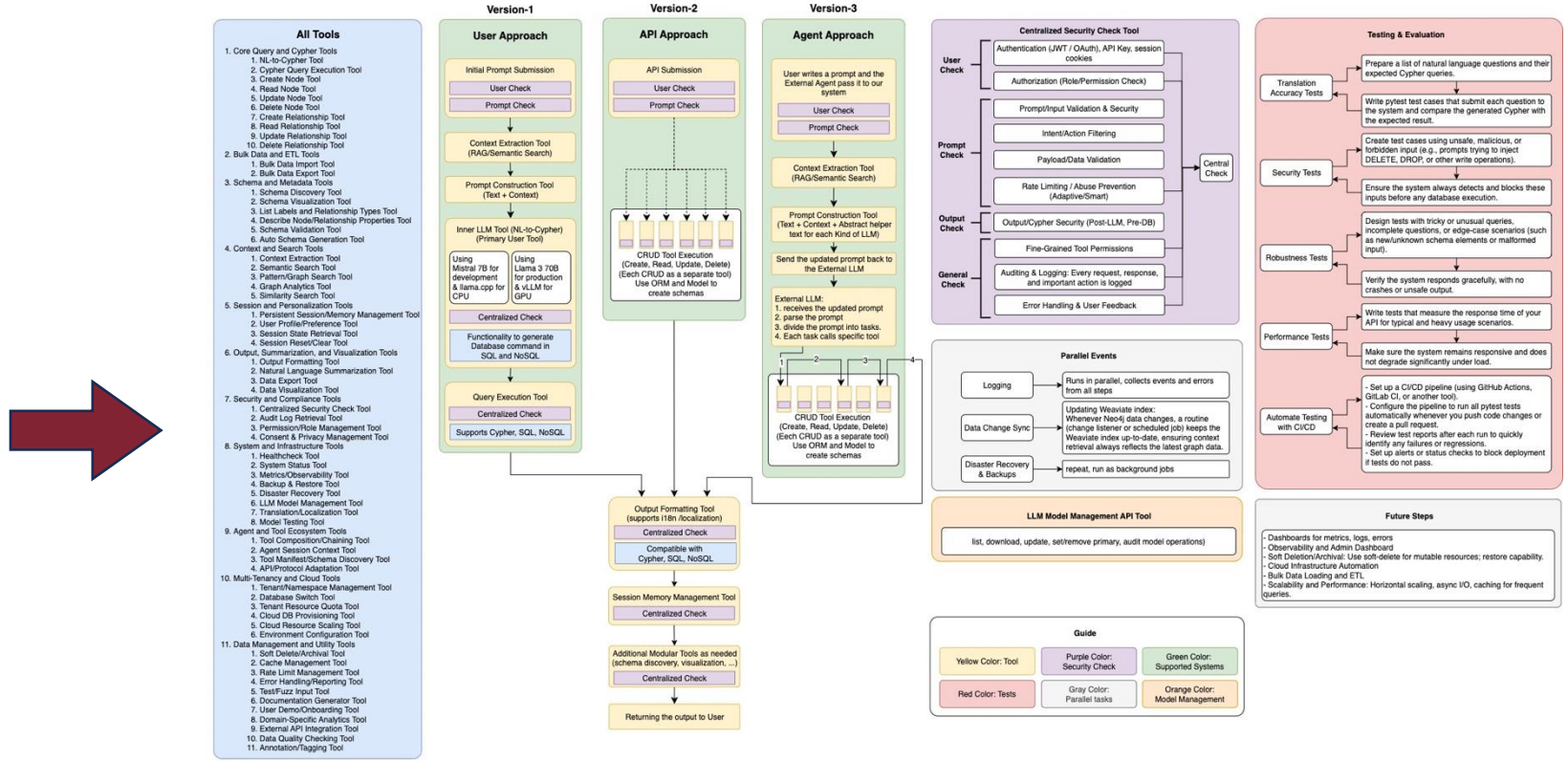
Why This Matters for CINECA

- **Remove developer bottlenecks**: researchers query graph data directly using natural language.
- **Secure and compliant**: RBAC, tenant isolation, and full audit logs ensure safe multi-tenant use and governance.
- **Scalable and reliable**: agent workflows and background jobs handle complex tasks at scale.
- **Reproducible results**: clear step tracking and outputs make results explainable.
- **Lower cost, full control**: run models locally (e.g., Ollama) to cut API costs and keep data inside CINECA.
- **Flexible and future-ready**: switch LLM providers (OpenAI, Azure, Ollama) without code changes—new models plug in easily.

ARCHITECTURE – FIRST DESIGNS

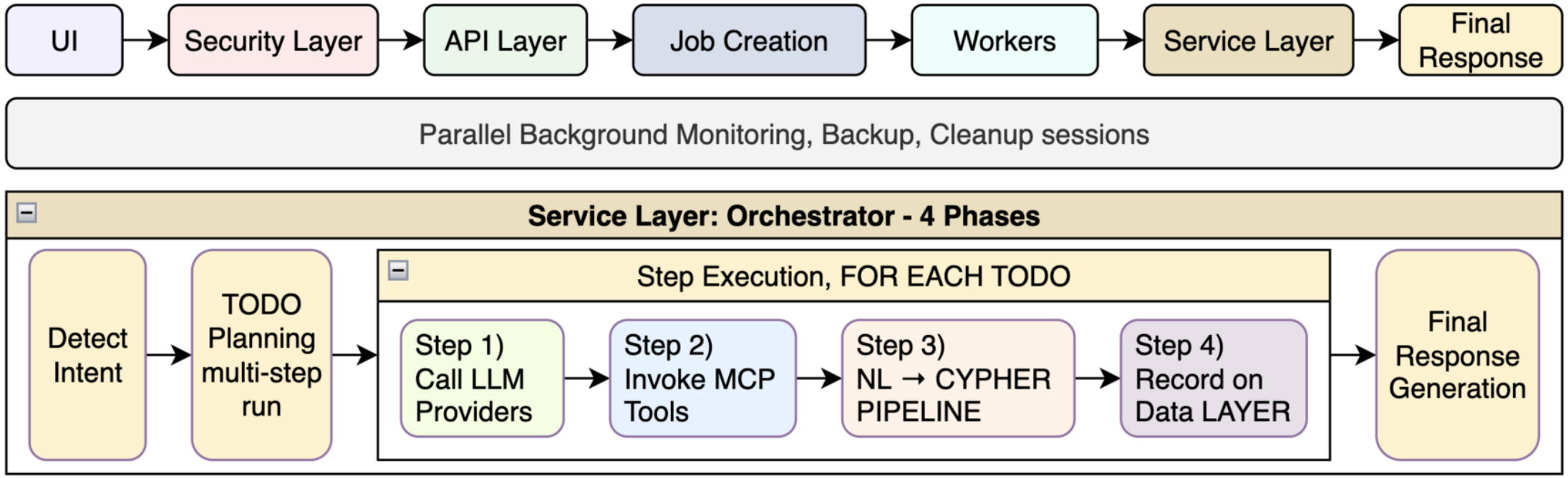
Showing Evolutions in Six months





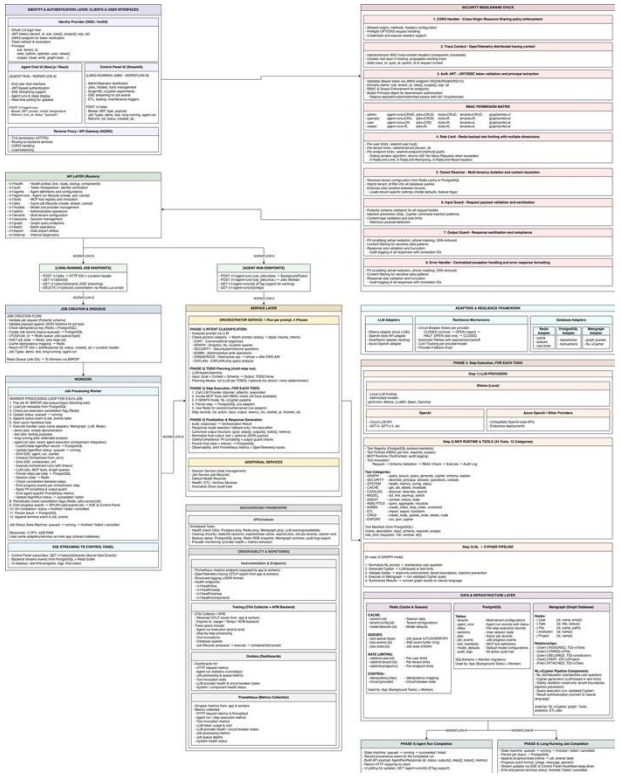
ARCHITECTURE - FINAL

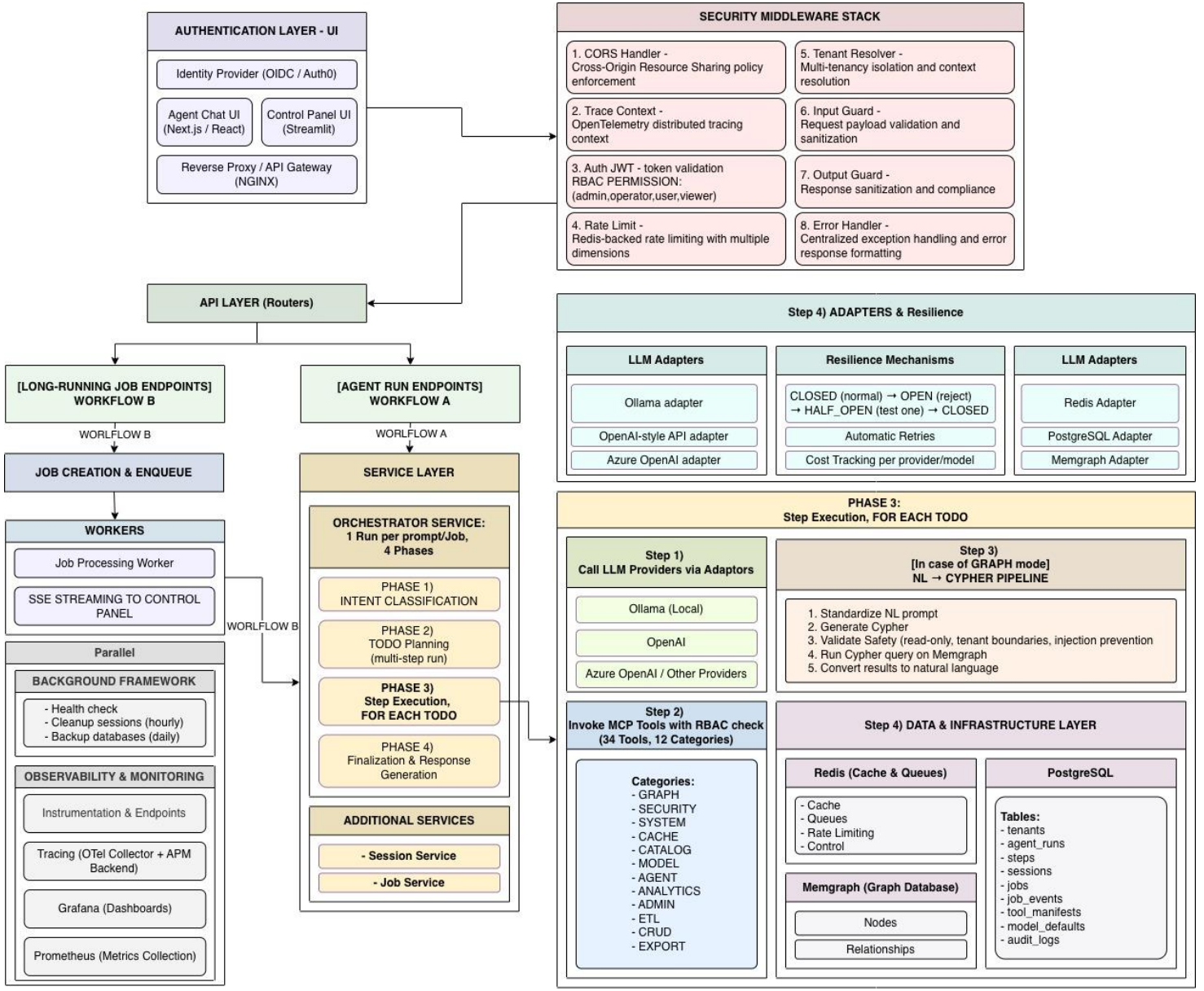
At a Glance



ARCHITECTURE - FINAL

Simplified





WHAT WAS DELIVERED

A complete, production-ready agentic platform for NL access to graph databases.

Full Stack Production-Grade Platform:

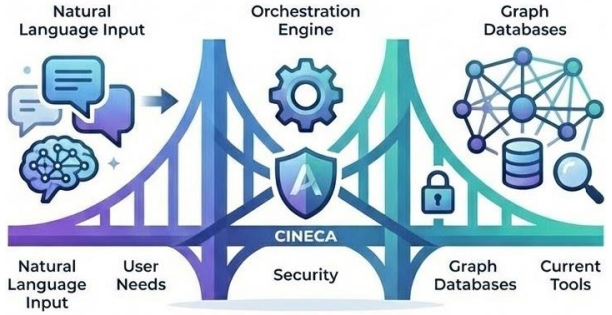
- End-to-end agentic system:s NL input → tool-executed, auditable, reproducible workflows.
- Chat UI (Next.js, JWT, SSE), Admin UI (Streamlit), NGINX (TLS, CORS), 75+ API endpoints, async workers (Redis + SSE).

Orchestration Engine:

- 4 phases: Intent → TODO Plan → LLM+Tool Steps → Safe Final Output.
- Graph Mode (NL→Cypher): GRAPH intent → Normalize → Lookup → Cypher → 6-level validation → Memgraph exec.

MCP + Tools:

- 34 tools, 12 categories (Graph, Model, ETL…), with RBAC, schema checks, audit logs, rate limits.



Security & Governance:

- JWT, RBAC, tenant filtering, rate limits, Pydantic guards, PII filter, error trace ID—multi-tenant safe.

Data Layer (3 DBs):

- Redis (cache, queues), PostgreSQL (jobs, logs, secrets), Memgraph (Cypher, lineage).

LLM Agnostic:

- Switch OpenAI / Azure / Ollama; provider per request/session/tenant. No lock-in, local/offline support.

Resilience + Background Jobs:

Retries, circuit breakers, provider fallback, APScheduler: cleanup, backups, health checks.

Observability: Traces (OpenTelemetry), Metrics (Prometheus), Dashboards (Grafana: jobs, tools, LLMs, health).

PROJECT SCALE

Proving the platform is production-grade, not a demo.

Key Stats

- 76 API endpoints across 16 FastAPI routers categories
- 34 implemented MCP tools across 17 capability categories
- 16+ infrastructure components (databases, queues, tracing, rate limits, gateways, etc.)
- 3,000+ automated tests (unit, integration, security, orchestration logic)
- ~411,700 total lines of code across all formats

Repo Composition (by file type)

- Docs: 500+
- Source code: 1,450+
- Tests: 780+
- Scripts: 100+
- Configs: 400+

Language Breakdown (by LOC)

- Python: ~57.8%
- Markdown: ~22.6%
- TeX: ~6.4%
- JSON/CSV/YAML: ~7.2%
- Other (Shell, TSX): ~6%

Test coverage includes:

- Unit tests (core logic, utilities, policies)
- Integration tests (DBs, LLM adapters, MCP runtime)
- Security tests (auth, RBAC, scopes, rate limits, guards)
- End-to-end and workflow tests (agent runs, jobs, streaming)Resilience and failure-mode tests (fallbacks, retries, breakers)

STAKEHOLDERS & USE CASES

Who Uses the Platform and What They Actually Do.

Stakeholders

- **Researchers / End Users** → Ask questions, trigger runs, trace outputs, reproduce results.
- **Admins / Operators** → Monitor runs and jobs, manage tools/models, enforce policies.
- **Security / Compliance Teams** → Audit tool usage, enforce scopes, verify controls.
- **Developers** → Add new tools, integrate LLM providers, extend APIs.

Core Use Cases (UC1–UC5)

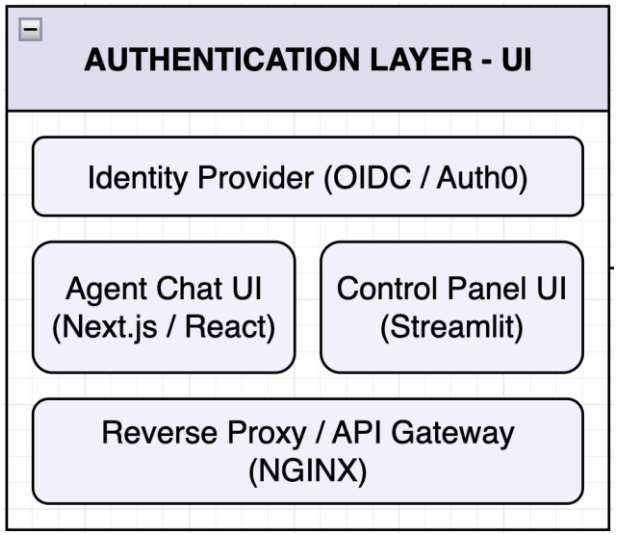
- **UC1: Chat/assistant runs**
  - Prompt an agent, receive a safe structured result.
- **UC2: Graph Q&A (NL→Cypher)**
  - Ask questions over Memgraph; validated Cypher is safely executed.
- **UC3: Background jobs / long tasks**
  - Run long processes with persistence, SSE progress updates, and cancellation support.
- **UC4: Monitoring & usage analytics**
  - Inspect job status, execution logs, tool usage metrics, and system health.
- **UC5: Tool discovery / controlled expansion**
  - Discover tools with schema and permissions, governed by tenant scope and roles.

AUTHENTICATION LAYER & UI

Users authenticate via Identity Provider, get JWTs, and use two UIs behind a secure reverse proxy.

Identity Provider - Users authenticate (OAuth 2.0):

- JWT tokens carry user identity and permissions:
  - **Sub:** unique user identifier | **tenant_id:** organization the user belongs to
  - **Roles:** what the user can do (admin, operator, user, viewer)
  - **Scopes:** {user:me, admin:all, tools:invoke:basic, tools:invoke:all, graph:read, graph:write}
- JWKS endpoint serves public keys for token checks; tokens support refresh and revocation.



Agent Chat UI (Next.js) - Workflow A

- End-user interface for chatting with AI agents | Authenticates users via JWT tokens
- Supports real-time streaming via Server-Sent Events (SSE)
- Displays agent run progress and individual steps, and status updates until completion
- **Input:** JWT, prompt, model, temp → **Output:** { run_id, status }

Control Panel UI (Streamlit) - Workflow B

- Admin and operator dashboard for platform management
- Manages jobs, models, tools, tenant, and streams job progress via SSE in real-time
- Triggers ETL pipelines, backups, and maintenance tasks
- **Input:** Bearer JWT, job type, payload, Job Types: demo, test, long-running, agent.run → **Output:** {id, status, created_at}

Reverse Proxy / API Gateway (NGINX)

- Terminates TLS/HTTPS connections securely, Routes incoming requests to appropriate backend services
- Handles CORS (Cross-Origin Resource Sharing) and Load balances traffic across multiple backend instances

Authentication: Users log in via OAuth 2.0 and receive a JWT with identity, tenant, roles, scopes. This token is verified on every API call.
Authorization: RBAC enforces access. Roles and scopes show what actions users can perform (viewer = read-only, admin = full access).

SECURITY MIDDLEWARE STACK (Part 1/3)

Every request passes through 8 Security middleware layers before reaching the business logic.

1. CORS Handler

- Configures allowed origins, methods, and headers for cross-origin requests
- Handles preflight OPTIONS requests automatically
- Supports credentials and custom expose headers

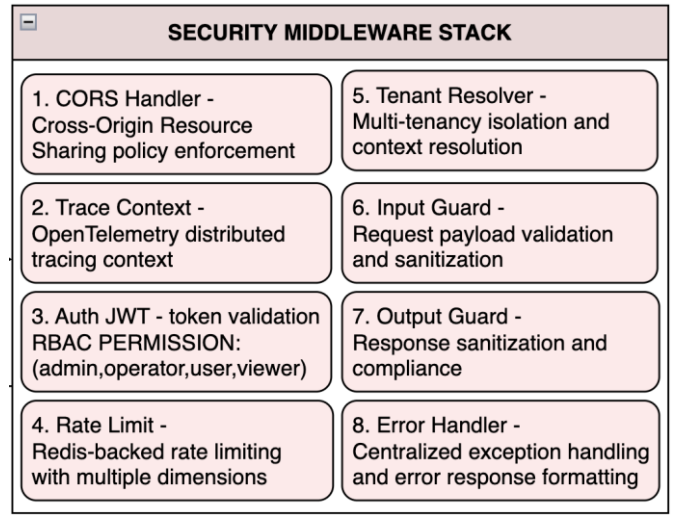
Example: A request from https://chat.cineca.it is allowed, but a request from https://malicious-site.com is blocked.

2. Trace Context (OpenTelemetry)

- Injects/extracts W3C trace headers (traceparent, tracestate)
- Creates root span if missing; propagates existing traces
- Adds trace_id, span_id, parent_id to request context

Example:

A user's chat request gets trace_id: abc123, which follows the request through the API, orchestrator, Memgraph, and back. All visible in Grafana.



Role	agent-runs	jobs	tools	tenants	Graph (write)
admin	CRUD	CRUD	CRUD	CRUD	✓
operator	CRU	CRUD	R	R	✗
user	CR	CRD	R	✗	✗
viewer	R	R	R	✗	✗

3. Auth JWT (OAuth 2.0) with RBAC

- Rejects invalid/expired tokens → 401 Unauthorized

Example: A viewer tries to delete an agent run → gets 403 Forbidden

4. Rate Limiter (Redis-backed)

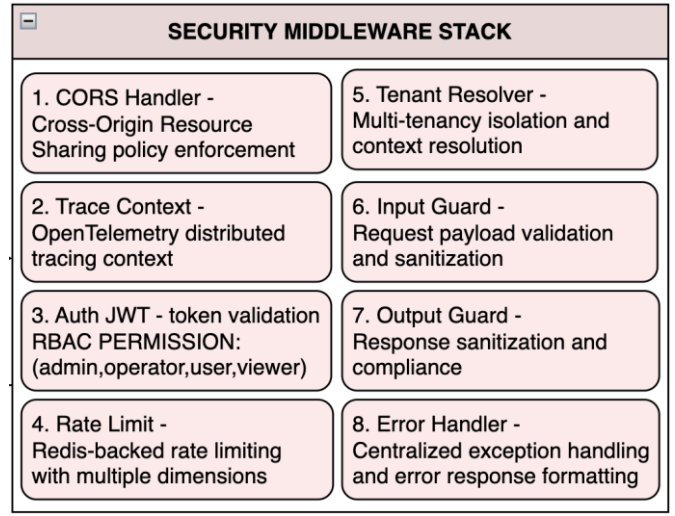
- Controls number of requests users can make.
- Returns 429 Too Many Requests when limits are exceeded. | Tracks limits at three levels: user, tenant, and endpoint
- Sends headers back the number of remaining requests → (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset)

Example:

User exceeds 100 requests/minute → receives 429 Too Many Requests with header X-RateLimit-Reset: 45 (seconds until reset).

SECURITY MIDDLEWARE STACK (Part 2/3)

Every request passes through 8 Security middleware layers before reaching the business logic.



5. Tenant Resolver

- Identifies which organization the request belongs to
- Loads tenant config from Redis cache (or PostgreSQL fallback)
- Automatically filters all database queries by tenant_id
- Ensures users can only access their own organization's data
- Applies tenant-specific settings. A config like model defaults

Example:

User from tenant_id: cineca_bioinformatics only sees bioinformatics ‘s data, and never data from other organizations.

6. Input Guard

- Validates all incoming data before it enters the system.
- Uses Pydantic schemas to verify data structure and types.
- **Applies injection attack detection by** scans for malicious patterns like:
  - **SQL injection** (DROP TABLE, UNION SELECT), **Cypher injection** (graphDB attacks), Shell commands (; rm -rf /)
- Checks request headers and limits oversized payloads that could crash the system
- Detects and matches unusual patterns or broken data structures that indicate potential attacks

Example:

A prompt containing "; DROP TABLE users;-- is detected as SQL injection and rejected with 400 Bad Request.

SECURITY MIDDLEWARE STACK (Part 3/3)

Every request passes through 8 Security middleware layers before reaching the business logic.

7. Output Guard

- Detect sensitive data before sending responses (emails, phones, SSNs)
- Filters patterns that might leak internal information
- Enforces maximum response size to prevent memory issues and truncates oversized responses.
- Logs all responses with correlation IDs for auditing

Example:

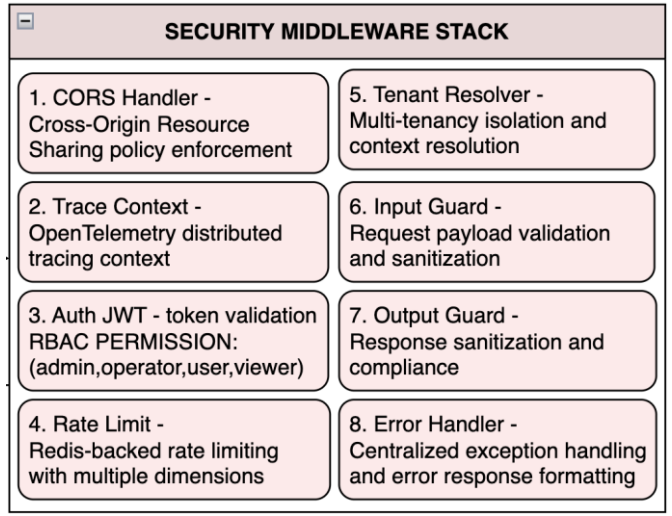
An LLM response containing user@email.com is automatically redacted to [EMAIL REDACTED] before reaching the client.

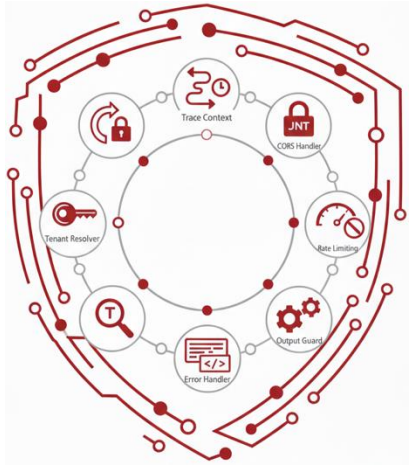
8. Error Handler

- Catches all exceptions in one place for consistent handling
- Returns standardized error responses across the API
- Hides internal details to prevent information leakage
- Attaches correlation IDs so errors can be traced in logs
- Records all errors for security auditing and debugging

Example:

A PostgreSQL connection timeout occurs → instead of exposing:
psycopg2.OperationalError: connection refused at 10.0.1.5:5432,
the client receives:
{"error": "Service temporarily unavailable", "correlation_id": "req-xyz789"}.





API LAYER - Routers

The platform exposes 75 endpoint routes organized into 16 versioned API groups, each for specific domain.

Endpoint	Description
/v1/health	**Health Probes** → Kubernetes-compatible endpoints for liveness (is the process running?) → readiness (can it serve requests?), and startup (has initialization completed?) → Includes per-component status for PostgreSQL, Redis, and Memgraph.
/v1/auth	**Authentication** → Token introspection and identity verification via OIDC/JWT claims.
/v1/agents	**Agent Configuration** → CRUD operations for agent definitions, prompt templates, and behavior settings.
/v1/tenants	**Multi-tenancy management** → create new tenant organizations, configure tenant-specific settings (default models, rate limits), set usage quotas, and enable/disable features per tenant
/v1/models	**LLM provider management** → register models, configure API keys, set default models per tenant
/v1/tools	**MCP tool registry** → Discover available tools, retrieve input schemas, and invoke tools with validated parameters.
/v1/jobs	**Manages long-running async jobs** → create jobs, stream real-time progress via SSE, and cancel running jobs
/v1/agent-runs	**Execution Lifecycle** → Controls agent execution lifecycle, start a new run, poll for status updates, retrieve results, or cancel mid-execution
/v1/graph	**Graph Database Interface** → Execute Cypher queries directly or via NL-to-Cypher translation.
/v1/sessions	**Conversation session management** → create sessions, store chat history, manage context windows
/v1/batch	**Batch operations for bulk processing** → process multiple agent runs/tool invocations in a single request
/v1/admin	**Administrative operations** → clear caches, database migrations, run maintenance tasks, view system stats
/v1/internal	**Internal diagnostics and debugging** → view internal state, check configuration, run diagnostics (restricted to admins)
/v1/export	**Data export** → download agent runs, job results, or audit logs in CSV or JSON format

API LAYER – Workflows A & B

The platform supports two execution paths for agent processing, each with dedicated API endpoints.

WORKFLOW A: Agent Run Endpoints	
POST /v1/agent-runs	**Start synchronous execution** → By default (use_jobs=false), runs in-process via FastAPI BackgroundTasks. Fast startup, but tied to the API server lifecycle.
POST /v1/agent-runs?use_jobs=true	**Start async execution** → Routes to the Jobs Worker for fault-tolerant processing. Survives API restarts and supports horizontal scaling.
GET /v1/agent-runs/{id}	**Retrieve run details** → Returns status, outputs, metrics, and TODO items. Supports ETag headers for efficient client-side caching and conditional requests.
GET /v1/agent-runs/{id}/steps	**Retrieve execution trace** → Returns the step-by-step breakdown: each LLM call, tool invocation, graph query, with inputs, outputs, and timing.

WORKFLOW B: Long-Running Job Endpoints	
POST /v1/jobs	**Create a new job** → Returns HTTP 202 Accepted immediately. Response includes a Location header pointing to the job resource for tracking. Job is enqueued to Redis for async processing.
GET /v1/jobs/{id}	**Retrieve job status** → Returns current state (queued, running, finished, failed, cancelled) along with metadata, timestamps, and results when complete.
GET /v1/jobs/{id}/events	**Stream real-time updates** → Opens an SSE (Server-Sent Events) connection to receive live progress updates, heartbeats, and final completion events as the job executes.
DELETE /v1/jobs/{id}	**Cancel a running job** → Sets a cancellation flag atomically via Redis Lua script. Worker checks this flag between steps and terminates gracefully.

JOB CREATION

Client submits a long-running task → system ensures reliability, idempotency, and prevents duplicates.

1. Request Validation
- Validates request against **Pydantic schema** (required fields, correct types)
- Validates payload against **job-type-specific JSON Schema**
- *Example:* agent.run requires { prompt, user_id, tenant_id }

2. Idempotency Check
- Looks up idempotency key in **Redis** (cache) and **PostgreSQL** (durable)
- If found → returns existing job response, preventing duplicates
- *Example:* Client retries after timeout → same job returned, not duplicated

3. Job Record Creation
- Creates new Job record in **PostgreSQL** with status = queued
- PostgreSQL = **source of truth** (jobs persists even if Redis is down)
- *Example:* INSERT INTO jobs (id, status, payload) VALUES (uuid, 'queued', {...})

Supported Job Types:
- **demo:** Simple queue testing | **test:** Validation purposes
- **long-running:** Extended batch processing
- **agent.run:** Full orchestration (LLM + tools + graph)

4. Redis Queue Enqueue
- **LPUSH** job ID to type-specific queue: jobs:queue:{type}
- Example: jobs:queue:agent.run for agent execution jobs
- Workers consume via **BRPOP** in FIFO order
- Job state is also cached via HSET at key jobs:state:{id} for fast access
- *Example:* LPUSH jobs:queue:agent.run "job-123" → HSET jobs:state:job-123 {...}

5. Idempotency Cache
- Stores idempotency key → job ID mapping in Redis to speed up future duplicate checks
- *Example:* SET idempotency:abc123 → job-123 (with TTL)

6. HTTP Response
- Returns **HTTP 202 Accepted** (acknowledging the job was accepted for processing, not completed)
- Response body: JobResponse { id, status: "queued", created_at }
- Includes a **Location header** pointing to /v1/jobs/{id} for status polling
- *Example:* 202 Accepted + Location: /v1/jobs/job-123

WORKERS (Job Processing Engine)

They are background processes that consume jobs from Redis queues, enabling fault-tolerant async execution.

1. Job Acquisition

- Worker blocks on BRPOP jobs:queue:{type} → efficiently waits for new jobs without CPU waste
- When a job ID arrives, worker has exclusive ownership (atomic pop guarantees no duplicates)
- *Example:* BRPOP jobs:queue:agent.run 0 → returns "job-123" when available

2. Load Job Metadata

- Fetches full job record from **PostgreSQL** (payload, type, tenant, user, timestamps)
- PostgreSQL is the source of truth;
- Redis queue only holds job IDs
- *Example:* SELECT * FROM jobs WHERE id = 'job-123'

3. Pre-Execution Cancellation Check

- Checks Redis key jobs:cancel:{id} before starting any work
- If cancellation was requested while job was still queued → skip execution, mark as cancelled.
- *Example:* User clicked "Cancel" in UI before worker picked up the job

4. Status Transition: queued → running

- Updates job status in PostgreSQL and Redis state cache
- Appends status_changed event to job_events table for audit trail
- Notifies SSE subscribers that execution has begun

5. Heartbeat Task

- Starts an async background task that periodically updates a Redis key (e.g., every 30s)
- Allows the system to detect **dead workers** (if heartbeat stops, job is considered stuck)
- *Example:* SETEX jobs:heartbeat:job-123 60 <timestamp> refreshed every 30s

6. Execute Job Handler

- Once the worker has acquired a job and updated its status, it routes the job to the correct **handler function** based on the job type.
- It supports Shared Codebase: The worker process runs the **exact same code** as the main API server. This means:
  - Same MemgraphClient for graph queries
  - Same LLMAdapter for calling language models
  - Same RedisClient for caching
  - Same security controls (PII scrubbing, output guards)

AGENT.RUN JOB HANDLER (Full Orchestration Pipeline)

Executes complete AI agent workflows: LLM reasoning, tool invocations, graph queries, and security processing.

1. Initialize
- Creates AgentRun record in PostgreSQL with status = running and links to job ID.
- e.g. { id: "run-456", job_id: "job-123", status: "running" }

2. Emit Startup Events
- Pushes SSE events (agent_run_started, orchestrator_init) so Control Panel shows immediate feedback
- Users see progress before any LLM call is made

3. Orchestrate
- Calls orchestrator.run(prompt, context, timeout) as the core execution engine.

4. Execute Steps
- For each TODO step: makes LLM calls, invokes MCP tools, runs Memgraph queries (NL-to-Cypher)
- Logs all actions (inputs, outputs, duration, token counts)

5. Persist Progress
- Saves completed steps immediately to PostgreSQL, caches session state in Redis
- Enables crash recovery, if worker dies, continue last saved step

6. Check Cancellation
- Between each step, worker checks Redis key jobs:cancel:{id}
- If set, worker stops after the current step (cooperative cancellation)
- Completed steps are saved; remaining steps are skipped
- Stopping mid-LLM-call would leave incomplete data

7. Emit Progress Events (UI displays a live progress)
- Each orchestration step triggers an SSE event
  - Like: { step: 3, action: "tool_call", tool: "graph.query" }
- Events pushed to Redis via RPUSH jobs:events:{id}

8. Post-Process
- **PIIScrubber:** Redacts personally identifiable information (emails, phones)
- **OutputGuard:** Filters patterns that might leak internal information

9. Emit Metrics (Enables dashboards, alerts, and cost tracking)
- Publishes Prometheus metrics: agent_run_duration_seconds, success/failure counters, TODO count, llm_tokens_total

10. Finalize
- Updates AgentRun.status (succeeded/failed), Saves result to PostgreSQL
- Emits terminal SSE event: orchestration_complete with final output
- Example: { status: "succeeded", response: "Found 5 institutions...", steps: 7, tokens: 2340 }

SERVICE LAYER - Orchestrator Service (Part 1/3)

The Service Layer contains business logic. the Orchestrator Service processes prompts through 4 phases.

PHASE 1: Intent Classification

- The orchestrator analyzes the incoming prompt using an LLM to determine the **intent category**. This classification drives which execution path is taken.

How it works:

- First checks if the prompt matches entries in a **prompt catalog** (predefined patterns)
- If no match, calls classify_intent() using the LLM to determine category

Intent Categories:

- **CHAT** → Conversational responses (*"What is ML?"*)
- **GRAPH** → NL-to-Cypher queries (*"Which institutions collaborated?"*)
- **SECURITY** → Permission/access questions (*"Who has admin access?"*)
- **ADMIN** → Administrative writes (*"Create tenant ACME"*)
- **DANGEROUS** → Destructive ops → refused, offers EXPLAIN instead (*"Delete all data"*)
- **EXPLAIN** → Query analysis without execution (*"Explain what delete would do"*)

PHASE 2: TODO Planning

- For multi-step tasks, the orchestrator uses LLM-based planning to decompose the goal into a **TODO list**, an ordered sequence of actions.

Inputs:

- User's goal (the original prompt)
- Conversation context (prior messages, session state)
- Schema information (available tools, graph schema, permissions)

Output:

- A TODO array:

[{ action: "query_graph", params: {...} }, { action: "summarize", params: {...} }]

Planning Modes:

- **full** → LLM generates a detailed plan for each TODO item. Best for complex, multi-step tasks.
  - *Example:* "Analyze collaboration patterns and generate a report" → 5-step TODO plan
- **optional** → Tries direct execution first; falls back to planning if needed. Good for simple-to-medium complexity.
  - *Example:* "What's the weather?" → direct response, no planning needed
- **none** → Deterministic execution without LLM planning. Used for predefined workflows or catalog-matched prompts.
  - *Example:* Matched catalog entry → fixed execution path

SERVICE LAYER - Orchestrator Service (Part 2/3)

The Service Layer contains business logic. the Orchestrator Service processes prompts through 4 phases.

PHASE 3: Step Execution

- Execute each TODO item, invoking LLMs, tools, and databases as needed.

For each step in the TODO plan, the orchestrator:

1. Call LLM Provider

- Uses specialized LLM roles depending on the step:
  - **Planner** → decides what to do next
  - **Reflector** → evaluates intermediate results
  - **Responder** → generates user-facing output
- LLM calls go through the resilient adapter layer (circuit breakers, retries, fallbacks)

2. Invoke MCP Tools

- The platform provides **34 MCP tools** across 17 categories
- Before invocation, **RBAC check** verifies the user has the required scope
- Tools include: graph queries, cache operations, security checks, data export, etc.
- *Example:* graph.secure_query tool invoked with generated Cypher

3. Graph Mode Pipeline (if applicable)

- For GRAPH intent, triggers the **NL-to-Cypher pipeline**:
  - Normalize natural language input
  - Generate Cypher query via LLM
  - Validate query safety (read-only, depth limits, tenant boundaries)
  - Execute on Memgraph
  - Build natural language response from results

4. Persist Step

- Each completed step is immediately saved to PostgreSQL via the database adapter
- Step record includes: { id, action, input, output, latency_ms, started_at, finished_at }
- Enables audit trail, crash recovery, and step-by-step debugging

5. Use Redis

- **Session state** → maintains conversation context across steps
- **Cache** → stores intermediate results for efficiency
- **Cancellation flag** → checked between steps for cooperative cancellation

SERVICE LAYER - Orchestrator Service (Part 3/3)

The Service Layer contains business logic. the Orchestrator Service processes prompts through 4 phases.

PHASE 4: Finalization & Response Generation: Assemble the final response with safety checks and observability.

1. Build Response

- Calls build_response() to aggregate results from all steps.
- Selects response mode based on execution outcome:
  - **fallback-only** → uses cached/default response if LLM failed
  - **llm-best-effort** → uses LLM-generated response with quality validation

2. Canonical Output Structure

```
{
"goal": "Find collaborating institutions",
"steps": [step1, step2, step3],
"outputs": ["Found 5 institutions…"],
"todos": [completed_todo1, completed_todo2],
"metrics": { "duration_ms": 2340, "tokens": 1850 }
}
```

3. Normalize Output

- Structures final output as text (for display) + optional JSON payload (for programmatic use)
- Ensures consistent format regardless of execution path

4. Safety & Compliance

- **PIIScrubber:** Redacts personally identifiable information (emails, phones)
- **OutputGuard:** Filters patterns that might leak internal information
- Both checks run on every response before it reaches the user

5. Persist Final State

- Saves complete agent run result to PostgreSQL (result_json column)
- Updates status to succeeded or failed
- Records final metrics for billing and analytics

6. Observability

- **Prometheus metrics** → emits counters, histograms, gauges for monitoring dashboards
- **OpenTelemetry traces** → distributed tracing spans for debugging and performance analysis
- Enables alerting, capacity planning, and SLA tracking

LLM PROVIDERS (Model-Agnostic Architecture)

The platform works with any OpenAI-compatible API — local or cloud.

Ollama (Local Hosting)

A local LLM inference server that runs open-weight models on your own hardware.

- **Data sovereignty** → prompts and responses never leave your infrastructure
- **No API costs** → only pay for compute resources
- **Offline capable** → works without internet connectivity
- **Customization** → fine-tune models for domain-specific tasks

Supported Models:

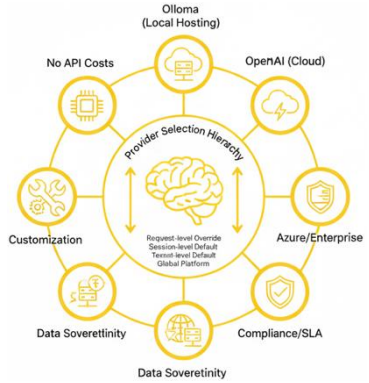
- **Phi-3 Mini** → Microsoft's small-but-capable model (3.8B parameters), optimized for reasoning
- **Mistral / Mixtral** → High-quality open models from Mistral AI, excellent reasoning-to-size ratio
- **LLaMA 2/3** → Meta's open foundation models (7B–70B), widely adopted baseline
- **Qwen** → Alibaba's multilingual models, strong on diverse languages
- **Gemma** → Google's lightweight models, optimized for efficiency

OpenAI (Cloud)

- **Hosted API** with state-of-the-art proprietary models
- **Benefits:** Zero infrastructure, automatic updates, rich features
- **Models:** GPT-4/4o/4 Turbo, GPT-3.5 Turbo

Azure OpenAI / Others

- **Enterprise-grade** OpenAI access via Azure or compatible providers
- **Benefits:** Compliance certs, regional data residency, SLA guarantees
- **Compatible:** Azure OpenAI, Anthropic Claude, vLLM, text-generation-inference



Provider Selection Hierarchy

Request-level override
↓ (if not set)
Session-level default
↓ (if not set)
Tenant-level default
↓ (if not set)
Global platform default

This allows:

- Users to override for specific requests
- Teams to set their preferred provider
- Platform admins to set organization-wide defaults

MCP RUNTIME & TOOLS - 34 Tools, 12 Categories - (Part 1/5)

Every Model Context Protocol (MCP) tool the agent can invoke is registered, validated, authorized, and audited.

Tool Registry → All tool definitions (manifests) stored in PostgreSQL (name, description, input schema, scopes, rate limits). Enables query available tools.
Tool Policies → RBAC per tool. JWT scopes checked before invocation → 403 if unauthorized.
MCP Runtime → ToolContext carries identity/permissions. All calls audited (inputs, outputs, caller, duration).

Tool Invocation Flow

User Request
↓
Input Structure Validation via JSON Schema
- If invalid input → 400 Bad Request
↓
RBAC Check (If caller has required scopes)
- If missing scope → 403 Forbidden
↓
Execute Tool (actual operation)
- If execution error → 500 with error details
↓
Log All Attempts, Even Failures
↓
Return Result

Tool Manifest (PostgreSQL Record)

```
{
  "name": "graph.secure_query", # Unique tool identifier
  "description": "Execute tenant-isolated Cypher query",
  "input_schema": { # JSON Schema for parameter validation
    "type": "object",
    "properties": {
      "cypher": { "type": "string" },
      "params": { "type": "object" }
    },
  "required": ["cypher"]
  },
  "required_scopes": ["graph:read"], # Authorization
  "rate_limit": { # Per-user, 100 requests per 60 seconds
    "requests": 100,
    "window": 60
  }
}
```

MCP RUNTIME & TOOLS - 34 Tools, 12 Categories - (Part 2/5)

Every Model Context Protocol (MCP) tool the agent can invoke is registered, validated, authorized, and audited.

1) GRAPH: Query and interact with Memgraph

Tool	Description	Example
query	Execute raw Cypher	MATCH (b:Blast) RETURN count(b) → Returns count of all Blast nodes
secure_query	Cypher with tenant isolation	MATCH (b:Blast)-[:OUTPUT]->(f:File) RETURN b, f LIMIT 10 → Only returns data for caller's tenant
generate_cypher	NL-to-Cypher translation	"How many Blast nodes are there?" → Generates: MATCH (b:Blast) RETURN count(b)
schema	Get graph schema	Returns: Nodes: [Blast, BlastDb, BlastedSeq, File], Relationships: [OUTPUT, INPUT]
explain	Query plan without execution	"Profile the query that finds top Blast by outdegree" → Returns EXPLAIN output only

2) SECURITY — Inspect permissions and access control

Tool	Description	Example
describe_principal	Show current identity	Returns: { user_id: "u-123", tenant: "cineca", roles: ["operator"], scopes: ["graph:read"] }
allowed_operations	List permitted actions	"Do I have permission to run write queries?" → Returns: { graph:read: ✓, graph:write: ✗, admin:all: ✗ }
validate	Check specific operation	"Can I delete Blast nodes?" → Returns: { allowed: false, reason: "Missing scope: graph:write" }

3) SYSTEM — Platform health and configuration

Tool	Description	Example
health	Component status	Returns: { postgres: "healthy", redis: "healthy", memgraph: "healthy" }
metrics	Current metrics	Returns: { active_jobs: 3, agent_runs_today: 127, llm_tokens_used: 45000 }
config	View configuration	Returns: { default_model: "gpt-4o", rate_limit: 100/min, tenant: "cineca" }
status	Overall status	Returns: { api: "running", workers: 2, queue_depth: 5 }

MCP RUNTIME & TOOLS - 34 Tools, 12 Categories - (Part 3/5)

Every Model Context Protocol (MCP) tool the agent can invoke is registered, validated, authorized, and audited.

4) CACHE — Redis cache operations

Tool	Description	Example
get	Read cached value	cache.get("blast_count") → Returns: 186 (cached from previous query)
set	Store value	cache.set("blast_count", 186, ttl=3600) → Caches result for 1 hour
delete	Remove key	cache.delete("blast_count") → Removes cached value
invalidate	Clear pattern	cache.invalidate("blast_*") → Clears all Blast-related cache entries

5) CATALOG — Tool and resource discovery

Tool	Description	Example
discover	List resources	Returns: { node_types: ["Blast", "BlastDb", "File"], tools: 34, models: 3 }
describe	Resource details	catalog.describe("Blast") → Returns properties: [blast_version, blasttype, dbname, status, ...]
search	Search catalogs	catalog.search("sequence") → Returns: [BlastedSeq, BlastSeq]

6) MODEL — LLM provider management

Tool	Description	Example
list	Available models	Returns: ["gpt-4o", "gpt-3.5-turbo", "phi3:mini (Ollama)"]
info	Model details	model.info("gpt-4o") → Returns: { context_window: 128k, cost: $0.01/1k tokens }
warmup	Pre-load model	model.warmup("phi3:mini") → Loads model into GPU memory for faster first response
switch	Change model	model.switch("gpt-3.5-turbo") → Sets as active model for this session

MCP RUNTIME & TOOLS - 34 Tools, 12 Categories - (Part 4/5)

Every Model Context Protocol (MCP) tool the agent can invoke is registered, validated, authorized, and audited.

7) AGENT — Agent state management

Tool	Description	Example
context	Current context	Returns last 5 messages + current graph schema + user permissions
history	Past interactions	Returns: [{ prompt: "Count Blast nodes", response: "186", timestamp: ... }]
state	Read/write state	agent.state.set("last_query_type", "graph") → Persists for session

8) ANALYTICS — Data analysis operations

Tool	Description	Example
query	Analytical query	"Which output type (File, BlastDb, BlastedSeq) is most frequently produced?" → Returns aggregated counts
aggregate	Compute aggregations	"Compute completeness ratio: Blast nodes having both blast_version and blasttype" → Returns: 78%
visualize	Generate viz data	Returns chart-ready JSON: { labels: ["File", "BlastDb"], values: [120, 66] }

9) ADMIN — Database administration (admin-only)

Tool	Description	Example
create_index	Add index	"Create an index on Blast(blast_version)" → CREATE INDEX ON :Blast(blast_version)
drop_index	Remove index	DROP INDEX ON :Blast(blast_version)
constraint	Add constraint	CREATE CONSTRAINT ON (b:Blast) ASSERT b.task_id IS UNIQUE

MCP RUNTIME & TOOLS - 34 Tools, 12 Categories - (Part 5/5)

Every Model Context Protocol (MCP) tool the agent can invoke is registered, validated, authorized, and audited.

10) ETL — Data import/export and transformation

Tool	Description	Example
import	Load data	etl.import("samples.csv", target="Sample") → Creates Sample nodes from CSV
export	Extract data	"Export the top 10 BlastedSeq with most inbound OUTPUT" → Returns downloadable dataset
transform	Transform data	"Set default value blast_version='N/A' for Blast where missing" → Bulk update

11) CRUD — Graph node operations

Tool	Description	Example
create_node	Create node	crud.create("Blast", { task_id: "task-001", status: "running" })
update_node	Update node	crud.update("Blast", { id: "blast-123" }, { status: "completed" })
delete_node	Delete node	"Delete Blast nodes with no OUTPUT edges" (admin-only, prompts for confirmation)

12) EXPORT — Output format conversions

Tool	Description	Example
csv	Export as CSV	Query results → blast_version,blasttype,status\n"2.12.0","blastn","success"\n...
json	Export as JSON	Query results → [{ "blast_version": "2.12.0", "blasttype": "blastn", ... }]
cypher	Export as Cypher	Nodes → CREATE (:Blast { blast_version: "2.12.0", blasttype: "blastn" })

NL-to-CYPHER PIPELINE (Graph Mode)

Once the orchestrator detects GRAPH intent, a 6-stage NL-to-Cypher pipeline is triggered.

6 Pipeline Stages

1. Normalize (Standardize input (Make all lowercase, remove fillers))
- *Example:* "Hey, how many Blast nodes??" → "count blast nodes"

2. Catalog Lookup (Check for pre-validated pattern match)
- If matched → use cached Cypher (fast, deterministic)
- If not → proceed to LLM

3. Generate Cypher (LLM generates query using schema context)
- *Example:*
- "Which BlastedSeq has the most inbound OUTPUT?" →
  LLM Generates →
  MATCH (b:Blast)-[:OUTPUT]->(s:BlastedSeq) RETURN s, count(b) ORDER BY count DESC LIMIT 10

4. Validate (6-Layer)
- Syntax check, Tenant boundary, Query depth, timeout, result size limits
- Read-only enforcement (blocks DELETE, SET unless admin)

5. Execute (Run on Memgraph via graph.secure_query)
- Tenant isolation enforced, timing metrics captured

6. Summarize
- LLM converts results to natural language + structured data

•*Example:*
- **Natural language:** "The BlastedSeq with the most connections is 'BlastedSeq-001' with 15 inbound OUTPUT relationships from Blast nodes."
- **Structured data:** { node: "BlastedSeq-001", count: 15 }

End-to-End Example

"Show 5 Blast pairs outputting to same BlastedSeq"

1. Normalize → "blast pairs same blastedseq limit 5"
2. Catalog → No match → LLM
3. Generate →
MATCH (b1:Blast)-[:OUTPUT]->(s)<-[:OUTPUT]-(b2:Blast) WHERE b1<>b2 LIMIT 5
4. Validate → ✓ Read-only ✓ Tenant ✓ Depth ✓ Timeout
5. Execute → Returns 5 pairs
6. Summarize →
"Found 5 Blast pairs sharing BlastedSeq targets..."

DATA LAYER – REDIS, POSTGRESQL, MEMGRAPH - (Part 1/4)

There are 3-database architecture: Redis for speed, PostgreSQL for durability, Memgraph for relationships.

Database Comparison

Aspect	Redis	PostgreSQL	Memgraph
Role	Speed layer	Durability layer	Relationship layer
Use Case	Cache, queues, rate limits	State, audit, config	Graph queries, lineage
Data Model	Key-value + structures	Relational tables	Property graph
Query	Commands (GET, SET, LPUSH)	SQL	Cypher
Persistence	Optional (RDB/AOF)	Always (WAL, ACID)	Always (WAL)
Consistency	Eventual	Strong (ACID)	Strong (ACID)
Scaling	Horizontal (cluster)	Vertical (replicas)	Vertical
Latency	Sub-ms	Low ms	Low ms (in-memory)
Transactions	Limited (Lua scripts)	Full ACID	Full ACID
Memory	In-memory	Disk + cache	In-memory + disk
Platform Keys	session:*, jobs:queue:*, ratelimit:*	agent_runs, jobs, audit_logs	:Blast, :File, [:OUTPUT]
Failure Mode	Data loss if not persisted	Safe (WAL)	Safe (WAL)

DATA LAYER – REDIS, POSTGRESQL, MEMGRAPH - (Part 2/4)

Redis for Speed (Cache & Queues): In-memory data store for caching, queueing, and real-time coordination.

Cache (Fast Lookups)

Key Pattern	Purpose	Example
session:{id}	Session data	User context, conversation history
tenant:config:{id}	Tenant settings	Rate limits, default models
model:defaults:{id}	Model preferences	Per-user/tenant model choices

Rate Limiting (Sliding Window)

Key Pattern	Purpose
ratelimit:user:{id}	Per-user quotas (e.g., 100 req/min)
ratelimit:tenant:{id}	Per-org quotas
ratelimit:endpoint:{path}	Per-endpoint throttling

Cache (Fast Lookups)

Key Pattern	Purpose	Operations
jobs:queue:{type}	Job queue	LPUSH to enqueue, BRPOP to consume
jobs:events:{id}	SSE event buffer	Ring buffer for progress streaming
jobs:state:{id}	Job state	HASH with status, timestamps, metadata

Control (System Coordination)

Key Pattern	Purpose
idempotency:{key}	Prevents duplicate job creation
circuit:{provider}	Circuit breaker state for LLM providers
jobs:cancel:{id}	Cancellation flag for cooperative shutdown

DATA LAYER – REDIS, POSTGRESQL, MEMGRAPH - (Part 3/4)

PostgreSQL for durability: Relational database for durable state, audit trails, and transactional operations.

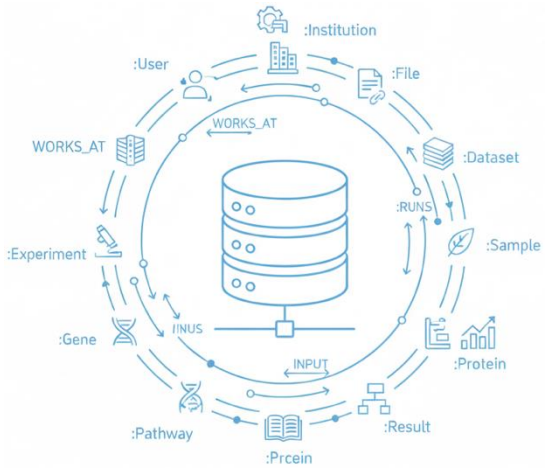
Core Tables

Table Names	Purpose	Key Fields	Details
tenants	Multi-tenant configs	id, name, admin_email, metadata	Tenant isolation. Unique lowercase name. Cascade deletes.
agent_runs	Execution records	id, session_id, status, result_json, todos, steps, metrics	Status: queued→running→succeeded/failed. ETags for caching.
agent_sessions	Conversation state	id, user_id, status, llm_preferences, tools, max_steps	Status: active→completed/cancelled. Configures tools & limits.
agent_steps	Per-step records	id, session_id, seq, type, input, output, status	Type: message/tool/system/error. Unique (session_id, seq).
jobs	Async job records	id, type, status, payload_json, result_json, idempotency_key	Status: queued→running→finished/failed. Idempotency + ETags.
job_events	Progress events	job_id, event_type, event_json, created_at	Append-only log. Used for SSE streams via to_sse_event().
tools	MCP tool definitions	name, version, input_schema, output_schema, owner_tenant_id	Versioned. JSON Schema validation. Unique (name, version).
tool_invocations	Tool execution audit	eid, tool_name, status, params_json, result_json, latency_ms	Status: pending→running→finished/failed. Idempotency support.
model_defaults	Default model selection	scope, tenant_id, instance_id	Hierarchy: request → session → tenant → global.
user_default_models	User model prefs	user_id, tenant_id, chat_instance_id	Per-user preferences. Unique (user_id, tenant_id).
providers	LLM provider registry	id, name, type, base_url, model, config_json	Type: openai_compatible/custom. Tenant or global scope.
provider_secrets	Encrypted API keys	provider_id, api_key_encrypted	Separate table. Encrypted at rest. 1:1 with provider.
audit_logs	Compliance trail	action, resource_type, resource_id, user_id, timestamp	Actions: create/update/delete. Indexed for fast queries.
idempotency_keys	Request dedup	key, owner_user_id, response_body, status_code	Prevents duplicates. Stores cached response for replay.

DATA LAYER – REDIS, POSTGRESQL, MEMGRAPH - (Part 4/4)

Memgraph (Graph Database): Property graph database for bioinformatics data with native Cypher support.

Node Types (14 types):



Node Label	Key Properties	Description
:User	user_id, firstName, lastName, email	Platform users
:Institution	id, name, country, type	Research organizations
:Task	task_id, status, start, tags	Computational jobs (Blast, CreateDb, Bold, etc.)
:File	file_id, user_filename, size, extension	Data files (Fasta, BlastDb, Xml, BlastedSeq)
:Dataset	id, name, description, version	Data collections
:Sample	id, name, organism, tissue	Biological samples
:Experiment	id, name, protocol, date	Lab experiments
:Publication	id, doi, title, journal, year	Research papers
:Gene	id, symbol, name, chromosome	Genetic sequences
:Protein	id, name, sequence, function	Protein data
:Pathway	id, name, description	Biological pathways
:Tool	id, name, version, type	Analysis tools
:Workflow	id, name, steps, inputs	Pipeline definitions
:Result	id, type, metrics, timestamp	Analysis outputs

Relationship Types (4 types):

Relationship	Pattern	Properties
WORKS_AT	(User)-[:WORKS_AT]->(Institution)	since, role, department
RUNS	(User)-[:RUNS]->(Task)	when user executes a task
INPUT	(File)-[:INPUT]->(Task)	file consumed by task
OUTPUT	(Task)-[:OUTPUT]->(File)	file produced by task

ADAPTERS

The platform connects to LLMs and databases through swappable adapters

LLM Adapters

- The platform supports multiple LLM providers through a unified adapter interface.
- Each adapter translates the platform's internal API to the provider's specific format.
- If you want to switch from OpenAI to Azure, you just swap the adapter. The rest of the code stays the same

Adapter	Provider	Use Case
Ollama	Local (Phi-3, Mistral, LLaMA)	Data sovereignty, no costs, offline
OpenAI	GPT-4, GPT-3.5	State-of-the-art, zero infra
Azure OpenAI	Azure-hosted	Enterprise compliance, SLA
Stub/Demo	Mock responses	Testing, CI/CD

Database Adapters

- Using adapters means the orchestrator code just calls cache.get() or graph.query() without knowing the underlying implementation details. If you switch from Redis to Memcached, you only change the adapter, not the business logic.

Adapter	Functions	Description
Redis	cache.get/set, queue.push/pop, ratelimit.check, lock.acquire	In-memory infrastructure for caching (TTL), async job queues, rate limiting, and distributed locks.
PostgreSQL	repository.CRUD, transaction.begin/commit, query.execute	Durable relational persistence: ORM-based CRUD, ACID transactions, and raw SQL for complex queries.
Memgraph	graph.query, graph.secure_query, graph.nl_to_cypher, graph.schema	Graph data layer: raw and tenant-isolated Cypher queries, NL→Cypher translation, and schema introspection for context.

RESILIENCE FRAMEWORK

The platform handles failures with backup plans instead of crashing.(Circuit breaker, Retries, Fallbacks)

Circuit Breaker: (Stop calling a broken service)

- If an LLM provider becomes unavailable, stops sending it requests. Each provider has independent circuit breaker state stored in Redis (circuit:{provider}).

States:

- **CLOSED:** Normal operation. Requests go through.
- **OPEN:** Service is broken. Requests are immediately rejected without trying. Returns fallback instead.
- **HALF-OPEN:** Waiting to test recovery. Allows one (or a few) request(s) through to check if the service is healthy again.

Example: If OpenAI returns 5 consecutive 500 errors, circuit opens → requests immediately fail-fast → after 60s, one test request allowed → if successful, circuit closes.

Retries:

- If a request fails once, try again with an exponential delay (This prevents hammering a recovering service)
- 1st attept → 0s, 2nd attempt → 1s, 3rd attempt → 2s, 4th attempt → 4s, 5th attempt → 8s, …

Provider Fallback Chain

- If the primary provider fails, the system automatically tries the next provider in the chain.
- Request → Primary Provider (e.g., OpenAI GPT-4) → Fallback 1 (e.g., Azure OpenAI GPT-4) → failure → Fallback 2 (e.g., Ollama Mistral)

Cost Tracking:

- Every LLM call is metered for billing and capacity planning.

Metric	What It Tracks	Why We Care
tokens_input	How many tokens in the prompt	Bigger prompts cost more
tokens_output	How many tokens in the response	Longer answers cost more
cost_usd	Estimated dollar cost	For billing and budgets
latency_ms	How long it took	For performance monitoring

BACKGROUND FRAMEWORK - APScheduler

The platform runs scheduled background tasks using APScheduler

APScheduler is a Python job scheduler that executes tasks at fixed intervals without manual intervention.

Task	Frequency	What It Does
Health Check	Every 30 seconds	Pings all critical services (PostgreSQL, Redis, Memgraph) to detect failures early. Also checks LLM providers are responsive and warms up models.
Cleanup	Every hour	Removes stale data to prevent database bloat: expired sessions, old cache entries, completed/failed jobs older than retention period, orphaned agent runs.
Backup	Daily	Creates recovery snapshots: PostgreSQL pg_dump, Redis RDB snapshot, Memgraph archives, and exports audit logs for compliance.
Provider Monitoring	Every 60 seconds	Checks LLM provider health, updates circuit breaker states, and emits metrics for dashboards.

OBSERVABILITY - (Part 1/2)

The platform provides full visibility into system behavior through metrics, traces, and logs.

Tracing (OpenTelemetry → APM Backend)

- Every request is traced end-to-end with distributed tracing using OpenTelemetry.

How it works:

- Request arrives → App creates trace span → Child spans for: LLM call, DB query, tool invocation → OTLP export to collector → Visualize in Jaeger / Tempo / Datadog

What It Captures:

Span	What It Captures
agent_run	Full execution from prompt to response
step_execution	Each TODO step within a run
tool_invocation	Every MCP tool call with inputs/outputs
db_query	PostgreSQL, Redis, Memgraph queries with timing
llm_call	LLM requests with model, tokens, latency
job_lifecycle	enqueue → execute → complete/fail/cancel

Why tracing?

- When a request is slow, you can see exactly where time was spent — was it the LLM? The database? A specific tool?

OBSERVABILITY - (Part 2/2)

The platform provides full visibility into system behavior through metrics, traces, and logs.

Prometheus

- Prometheus scrapes metrics from the app and workers every 15 seconds.

Metric Category	Examples
HTTP	http_request_duration_seconds, http_requests_total (by endpoint, method, status)
Agent Runs	agent_run_duration_seconds, agent_run_total (by status: success/failed)
Steps	agent_step_duration_seconds, agent_step_total (by type: tool/llm/graph)
Tools	tool_invocation_duration_seconds, tool_invocation_total (by tool name)
LLM	llm_tokens_total, llm_cost_usd_total, llm_latency_seconds (by provider/model)
Circuit Breaker	circuit_breaker_state (0=closed, 1=open, 0.5=half-open)
Jobs	job_queue_depth, job_processing_duration_seconds, job_total (by status)
System	component_health (1=healthy, 0=unhealthy per component)

Grafana:

- Pre-built Grafana dashboards visualize all metrics

Dashboard	Shows
HTTP Overview	Request rate, latency percentiles (p50/p95/p99), error rate
Agent Runs	Runs per minute, success rate, average duration, step breakdown
Job Processing	Queue depth, processing time, completion rate, failures
Tool Invocations	Most-used tools, latency by tool, error rates
LLM Providers	Provider health, circuit breaker states, token usage, costs
System Health	Component status (green/red), database connection pools, memory usage

PHASE 4: COMPLETION - Agent Run (Workflow A)

For synchronous agent runs that execute in-process via BackgroundTasks.

State Machine:

queued → running → succeeded/failed

- queued: Run created, waiting to start
- running: Orchestrator is actively processing
- succeeded: Completed successfully with output
- failed: Error occurred during execution

Completion Steps:

1. Log and store the completion event with timestamp, duration, and outcome in agent_runs table.
2. Build API Payload
3. Return HTTP Response immediately → Status code: 200 OK (success) or 500 (failure)
4. Client Polling (if needed) → client polls GET /agent-runs/{id}

Build API Payload:

```
{
  "id": "run-abc123",
  "status": "succeeded",
  "outputs": ["Found 5 institutions collaborating on..."],
  "steps": [
    { "type": "llm", "action": "classify_intent", "latency_ms": 340 },
    { "type": "tool", "action": "graph.query", "latency_ms": 120 }
  ],
  "todos": [
    { "action": "query_graph", "status": "completed" },
    { "action": "summarize", "status": "completed" }
  ],
  "metrics": {
    "duration_ms": 2340,
    "tokens_input": 850,
    "tokens_output": 420
  }
}
```

PHASE 4: COMPLETION - Long-Running Job Completion (Workflow B) - (Part 1/2)

For async jobs that run in a separate worker process.

State Machine:

queued → running → finished/failed/cancelled

queued: Job in Redis queue, waiting for worker

- running: Worker picked it up, actively processing
- finished: Completed successfully
- failed: Error occurred
- cancelled: User requested cancellation

Completion Steps:

1. Update PostgreSQL jobs table:

```
{status = 'finished',
completed_at = NOW(),
result_json = '{"response": "Found 5 institutions...", "tokens": 1850}'}
```

2. Record Events: Every state is recorded as an event in job_events table. Events are append-only (never deleted) for compliance

seq_id	event_type	event_json	created_at
1	queued	{"queue": "agent.run"}	10:00:00
2	started	{"worker": "worker-1"}	10:00:02
3	progress	{"step": 1, "message": "Classifying intent"}	10:00:03
4	progress	{"step": 2, "message": "Executing graph query"}	10:00:05
5	progress	{"step": 3, "message": "Generating response"}	10:00:08
6	complete	{"status": "finished", "duration_ms": 8000}	10:00:10

PHASE 4: COMPLETION - Long-Running Job Completion (Workflow B) - (Part 2/2)

For async jobs that run in a separate worker process.

3. Progress Event Format

- stage: Which phase: initialization, orchestration, finalization
- message: Human-readable description of current action
- percent: Progress bar value (0-100) for the UI

```
{"stage": "orchestration",
"message": "Executing step 3 of 5: graph.query",
"percent": 60}
```

4. Stream Updates via Server-Sent Events (SSE),

- SSE is a way for the server to push updates to the client in real-time (one-way communication).
- Server sends empty ping every 15s, to prevent connection timeout. Client displays progress bar and step-by-step log

How it works:

```
Client opens: GET /v1/jobs/{id}/events
Server sends:
event: started
data: {"status": "running"}
event: progress
data: {"message": "Step 2 of 5", "percent": 40}
event: progress
data: {"message": "Step 5 of 5", "percent": 95}
event: complete
data: {"status": "finished", "result": {...}}
```

5. Emit Terminal Status - Client knows to close the SSE connection and stop polling/listening

- Final SSE event: { "event": "complete", "status": "finished", "result": {...}

CONCLUSION

Closed the production gap

Closed the production gap:
- NL → Cypher with safe, auditable, reproducible workflows, no dev bottleneck, no black-box risk.

Full enterprise stack delivered:
- Secure UIs, gateway, orchestration, workers, and DBs, built for CINECA's multi-tenant ops.

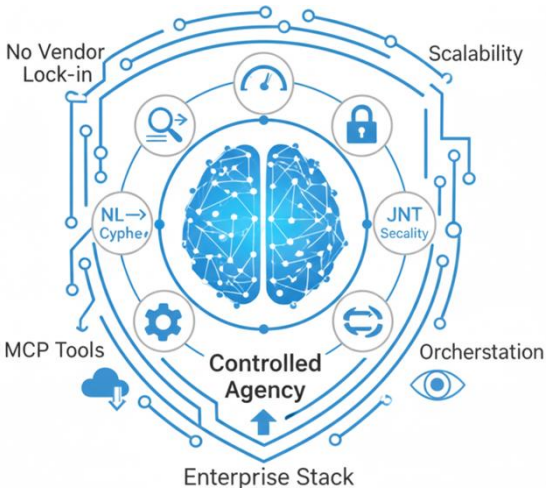
Controlled agency, not chat:
- Agent runs plan & execute via MCP tools, with RBAC, tenant isolation, I/O guards, and safe NL→Cypher (normalize → lookup → gen → 6-check → exec → summarize).

Production-ready at scale:
- 76 FastAPI endpoints, 34 tools, Redis + Postgres + Graph DB, auto-tests, retries, fallbacks, circuit breakers.

Built-in observability:
- Tracing (OpenTelemetry), metrics (Prometheus), dashboards (Grafana), for debugging, auditing, and cost tracking.

No vendor lock-in:
- Plug in OpenAI, Azure, Ollama; select per request/session/tenant, supporting sovereignty & cost control.

Key contribution:
- A secure, extensible NL-to-action platform, ready for tools/models/tenants expansion at Sapienza & CINECA.



COMPARISON WITH SOTA

Capability-by-capability comparison of your CINECA Agentic Platform against Top-10 similar platforms

Platform	Full Stack (UI + API + Jobs)	Orchestration Durability (Retries, Resume)	Agent Planning Loop (Multi-Step Tools)	Tool Ecosystem & Schema Controls	Security & Governance (RBAC, Audit)	Graph NL→Cypher Support (Secure + Tenant-Aware)	Observability (Tracing, Metrics, LLM Logs)	LLM-Agnostic (Model Provider Independent)	License / Pricing	Typical Limitation Compared to CAP	Fit Score
CINECA Agentic Platform	✓ Full-stack: UI + API + Jobs	✓ Durable: retries + checkpoints	✓ Built-in MCP loop engine	✓ Native registry + schema audit	✓ JWT, RBAC, tenancy, I/O guards	✓ Built-in NL→Cypher with validation & tenancy	✓ Full telemetry + LLM evals	✓ Yes (OpenAI, Ollama, etc.)	Custom-built (Internal only)	—	★ 5.0
Temporal	✗ Engine only (no UI/tools)	★ **Best-in-class workflow durability**	✗ No agentic planning loop	~ Pluginable tools (not MCP-native)	✓ Fine-grained RBAC + auth options	✗ No graph layer	✓ Metrics + traces	✓ Yes	Free OSS (MIT) + Paid Cloud	No agentic loop or UI; not a full platform	★★ 4.0
Argo Workflows	~ K8s-native stack	✓ DAGs + Retry Semantics	✗ No agent loop	~ Container steps only	~ Basic RBAC via K8s	✗ No Cypher / graph features	~ Limited metrics	✓ Yes	Free OSS (Apache-2.0)	K8s-focused; lacks planning & tool UX	★★ 3.5
LangGraph	✗ Library only	✓ Durable w/ state checkpointing	✓ Agent loop with state machine model	✓ Built-in agent tool patterns	✗ No RBAC / audit; DIY needed	~ Custom NL→Cypher possible	~ Partial (via adapters)	✓ Yes	Free OSS (MIT)	No orchestration stack; governance must be added manually	★★ 4.0
OpenAI Agents SDK	✗ SDK only	~ Runtime-level retry only	✓ SDK-defined agent planning	✓ Typed tools as functions	✗ No RBAC or tenancy	~ Custom Cypher interface possible	~ Logs via app code	~ Partial (OpenAI-focused)	Free OSS (MIT) + Paid OpenAI API	No job system, no multi-tenant security	★★ 3.5
Semantic Kernel	✗ Middleware only	~ Retry via plugins	✓ Plugin-based agent orchestration	✓ Tools + DI + planner interfaces	~ Basic auth via host app	~ Requires custom NL→Cypher logic	~ Custom logs via adapters	✓ Yes	Free OSS (MIT)	Middleware only; lacks orchestration/runtime infra	★★ 3.5
LlamaIndex	✗ RAG/agent SDK	~ Partial retries	✓ Tool-using planner + query agents	✓ Strong RAG support	✗ No RBAC, audit, tenancy	~ External Cypher logic possible	~ Logs via app code	✓ Yes	Free OSS (MIT) + Paid Cloud	No durability, security, or orchestration stack	★★ 3.0
Haystack	✗ Library only	~ Retryable components	✓ Agent + tools (non-durable)	✓ Tool abstraction + DSL	✗ No auth/governance	~ Cypher possible w/ custom nodes	~ Basic logs	✓ Yes	Free OSS (Apache-2.0)	Not production-grade orchestration	★★ 3.0
n8n	✓ GUI product + workflow UI	✓ Retry + error steps	~ Linear tool invocation (not agentic)	✓ Large integration catalog	~ Role-based app-level security	✗ Not graph-native	~ Workflow logs; no LLM eval	✓ Yes	Free OSS (SUL) + Paid Cloud SaaS	Good for automation; lacks agentic structure	★★ 2.5
Windmill	✓ UI + job scripting platform	✓ Durable jobs + cron	✗ No agent loop	✓ Script-based tool definitions	✓ RBAC, SSO, auditing	✗ Not graph-native	~ Run logs and dashboards	~ Partial (not LLM-centric)	Free OSS (AGPL mix) + Paid tiers	Good for internal workflows; lacks agent stack	★★ 3.5
Langfuse	✗ Observability layer only	✗ No execution	✗ No planning or tool use	~ Eval schema for prompts/tools	~ Some logging support	✗ Not applicable	✓ Best-in-class for LLM tracing	✓ Yes	Free OSS (MIT Core) + Paid SaaS	Tracing/monitoring only; no workflow or orchestration	★★ 3.0

FUTURE WORKS

Make Orchestration More Autonomous and More Reliable

Continue at CINECA (fixed-term contract):

- per latest HR update, keep improving, scaling, and deploying the platform across other CINECA HPC software environments.
- Alternatively, contribute to other AI-related areas aligned with CINECA priorities, depending on team needs and preferred direction.

Unify Workflows A + B (single lifecycle; B as default)

- Merge both into one execution path; use Workflow B (jobs + workers) as the base because it's durable, fault-tolerant, scalable, SSE-native. Keep a "sync-like" UX by streaming job events immediately (no in-process execution).

Add "Ask Mode" (non-agentic Q&A / data retrieval)

- Lightweight path: no TODO decomposition, but still enforce RBAC, tenant isolation, validation, auditing.

Strengthen autonomy and reliability in the orchestration engine:

- Handle more prompt types reliably (easy → hard, clean → messy)
  - Example (easy): "How many Blast nodes are there?"
  - Example (ambiguous): "Show top collaborations." → ask "top by what?" or apply a safe default.
  - Example (multi-intent): "Find top institutions and export to CSV." → plan: query → export.
  - Example (risky): "Delete failed jobs." → refuse or require admin + confirmation.
- Recover and reproduce runs (checkpointing + deterministic re-runs)
  - Example: worker crashes after step 2/5 → resume from step 3, not restart.
- Smarter scheduling under load (priorities + quotas + caps + backpressure)
  - Example: Prioritize quick interactive chat jobs over long ETL jobs.
  - Example: Tenant A max 20 concurrent jobs; Tenant B max 5.

Thanks For Your Attention